

Name:- Syed Jasim Us Sani

Roll no:- B-51

Branch:- AI&DS(B)

Batch:- B3

Aim:- Use sequence data types and their associated operations in Python a. Strings
b. Lists c. Arrays d. Tuples e. Dictionary f. Sets g. Range

Theory

1. What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types? Sequence data types in Python are collections that store multiple elements in an ordered manner, where each element is associated with a specific position called an index. These data types allow easy access, iteration, and manipulation of elements. The main sequence data types in Python are lists, tuples, and strings. A list is a mutable sequence, meaning its elements can be modified after creation. A tuple is immutable, so its elements cannot be changed once defined. A string is also immutable and is used to store sequences of characters. One important feature of sequence types is indexing. Indexing allows accessing individual elements using their position. In Python, indexing starts from 0. For example, in a list [10, 20, 30], the element 10 is at index 0. Negative indexing is also supported, where -1 refers to the last element. Another key feature is slicing, which allows extracting a portion of the sequence. Slicing uses the syntax `sequence[start:stop:step]`. The start index is inclusive, while the stop index is exclusive. This feature works consistently across lists, tuples, and strings, making it very powerful for handling subsequences.
2. What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications?

Lists and tuples are both sequence data types, but they differ mainly in terms of mutability. A list is mutable, meaning its elements can be changed, added, or removed after creation. In contrast, a tuple is immutable, meaning once it is created, its contents cannot be modified. Lists use square brackets `[ ]`, whereas tuples use parentheses `( )`. Lists are generally slower than tuples because they allow dynamic changes, while tuples are faster due to their fixed nature. In real-world applications, tuples are preferred when the data should remain constant throughout the program. For example, coordinates, configuration settings, or database records are often

stored as tuples to prevent accidental modification. Tuples are also used when data needs to be used as keys in dictionaries, as only immutable types can serve as keys.

3. How are Python arrays different from lists? When should you use the array module instead of a list?

In Python, lists are the most commonly used data structures and can store elements of different data types within the same collection. They are flexible and easy to use. Arrays, on the other hand, are provided by the array module and are more restricted. They can store only elements of the same data type, such as integers or floats. Because of this restriction, arrays are more memory-efficient and slightly faster when dealing with large numerical datasets. Lists consume more memory because they store references to objects, whereas arrays store actual values in a more compact form. Arrays are typically used in situations where performance and memory efficiency are important, such as numerical computations or when handling large datasets of uniform type. In most general-purpose programming tasks, lists are preferred due to their flexibility. Arrays are chosen when working with homogeneous data and when optimization is required.

4. Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples?

A dictionary in Python is a collection of key-value pairs, where each key is unique and is used to access its corresponding value. Dictionaries are implemented using a data structure called a hash table. When a key-value pair is added to a dictionary, Python uses a hash function to convert the key into a unique hash value. This hash value determines the index where the value will be stored in memory. When accessing a value, Python again computes the hash of the key and directly retrieves the value from the corresponding location. This mechanism makes dictionary operations such as lookup, insertion, and deletion very fast, with an average time complexity of $O(1)$. In contrast, lists and tuples require sequential searching ($O(n)$) to find an element because they do not use hashing. This is why dictionaries are significantly faster for lookups, especially when dealing with large datasets.

5. What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Sets and lists are both used to store collections of elements, but they differ in several important ways. A list is an ordered collection that allows duplicate elements and supports indexing. A set, however, is an unordered collection that does not allow duplicate values. When duplicate elements are added to a set, Python automatically removes them, ensuring that all elements remain unique. This is because sets are also implemented using hashing, similar to dictionaries. Sets do not support indexing or slicing because they do not maintain order. However, they are highly efficient for operations like membership testing and mathematical set operations such as union,

intersection, and difference. In practical scenarios, sets are useful when you need to remove duplicates from data, check whether an element exists in a collection quickly, or perform operations involving multiple datasets. Lists, on the other hand, are used when order and duplicates are important

```
#String Operation
print("\n String Operation ")
s="Hello Python"
print("Original String:",s)
print("Upper Case:",s.upper())
print("Lower Case:",s.lower())
print("Length:",len(s))
print("substring:",s[0:5])

#List Operation
print("\n List Operation ")
list1 = [10,20,30,40]
print("original list:",list1)
list1.append(50)
print("After append:",list1)
list1.remove(20)
print("After remove:",list1)
print ("List slicing:",list1[1:3])

#Array Operation
print("\n Array Operation ")
from array import *
arr=array('i',[1,2,3,4,5])
print("Array :",arr)
arr.append(6)
print("after append:",arr)
arr.remove(3)
print("after remove:",arr)

#Tuple operation
print("\n Tuple Operation ")
tup=(100,200,300,400)
print("tuple:",tup)
print("tuple length:",len(tup))
print("tuple slicing:",tup[1:3])

#Dictionary
print("\n Dictionary Operation ")
dict1={"name":"NILESH","age":20,"city":"sambhaji nagar"}
print("Dictionary:",dict1)
dict1["age"]=21
print("after update:",dict1)
print("keys:",dict1.keys())
print("values:",dict1.values())

#set operation
print("\n Set Operation ")
set1={1,2,3,4}
```

```
print("set:",set1)
set1.add(5)
print("after add:",set1)
set1.remove(2)
print("after remove:",set1)
```

```
#range function
print("\n Range Function ")
for i in range(1,6):
    print(i)
```

```
String Operation
Original String: Hello Python
Upper Case: HELLO PYTHON
Lower Case: hello python
Length: 12
substring: Hello
```

```
List Operation
original list: [10, 20, 30, 40]
After append: [10, 20, 30, 40, 50]
After remove: [10, 30, 40, 50]
List slicing: [30, 40]
```

```
Array Operation
Array : array('i', [1, 2, 3, 4, 5])
after append: array('i', [1, 2, 3, 4, 5, 6])
after remove: array('i', [1, 2, 4, 5, 6])
```

```
Tuple Operation
tuple: (100, 200, 300, 400)
tuple length: 4
tuple slicing: (200, 300)
```

```
Dictionary Operation
Dictionary: {'name': 'NILESH', 'age': 20, 'city': 'sambhaji nagar'}
after update: {'name': 'NILESH', 'age': 21, 'city': 'sambhaji nagar'}
keys: dict_keys(['name', 'age', 'city'])
values: dict_values(['NILESH', 21, 'sambhaji nagar'])
```

```
Set Operation
set: {1, 2, 3, 4}
after add: {1, 2, 3, 4, 5}
after remove: {1, 3, 4, 5}
```

```
Range Function
1
2
3
4
5
```

Conclusion

In this practical, we studied various Python data structures such as lists, tuples, arrays, dictionaries, and sets. We understood their characteristics, differences, and

practical applications. Lists provide flexibility due to their mutability, while tuples ensure data safety with immutability. Arrays are useful for handling large homogeneous data efficiently. Dictionaries offer fast data access using key-value mapping through hashing, and sets help in managing unique elements and performing mathematical operations. Overall, this practical enhanced our understanding of how different data structures work internally and when to use each of them effectively in real-world programming scenarios.